KÄYRÄ

What is the undocumented controller doing?

Found from data a site already logs. No vendor, no manual, no bump test, nothing written to the controller.

AMD DEVELOPER HACKATHON ACT III

TRACK 1 INTELLIGENT INDUSTRY

The problem

- Factories and buildings run on controllers nobody has documentation for. The integrator left, the vendor is gone.
- Nobody dares to retune, migrate or replace them, because nobody knows what they do together.
- The usual way to find out is a bump test: step the process and watch. It costs production time and nerve.
- I work as director of IT security and architecture in factories full of legacy OT. This is a problem I see at work.

Test bench: a heating stack nobody documented

District heating substation, two heating loops, one commercial controller. Its outputs are not logged. Käyrä sees outdoor temperature in and room temperatures out, and identifies the controller through the building.

OUTDOOR (FMI UTTI)

4486

hours, 2026-04-13 to 2026-10-17

ROOMS (HOME ASSISTANT)

3

4486 hourly means each

OUTDOOR RANGE

-1.6 / 28.8

°C, winter unobserved

The same engine runs on any process: sources, signals and groups are configuration (config/process.json). The house is the proof of concept.

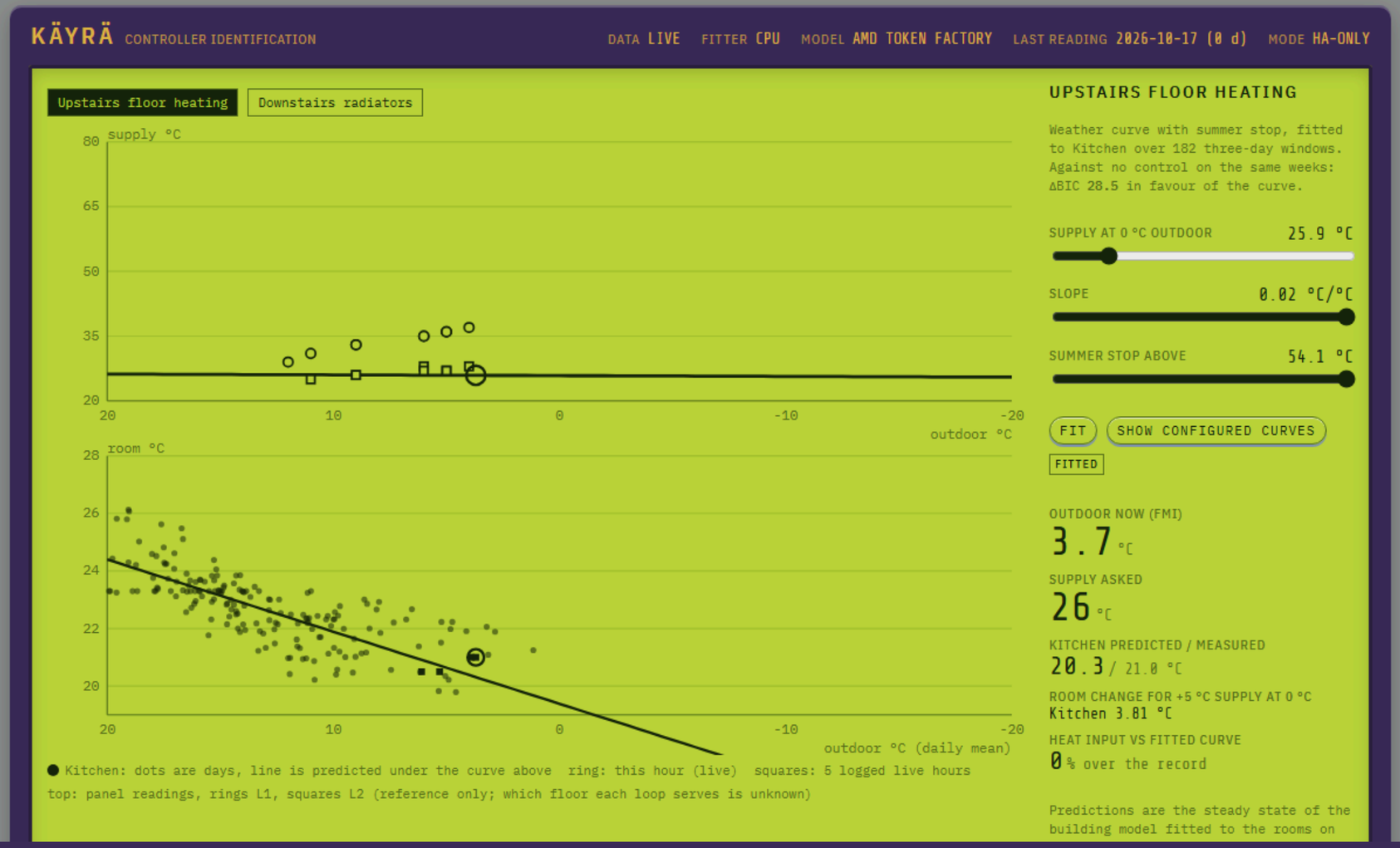
How: hypotheses as programs, values from the fitter

Every hypothesis is a short program with no numbers in it:

```
where(ema(T_out, tau) < cutoff, level + slope * T_out, OFF)
```

- The fitter finds every value: thousands of parameter sets simulated together as tensors (PyTorch, ROCm-ready), scored on held-out weeks.
- A language model on AMD reads what the best program fails to explain and writes new programs. The validator rejects any literal.
- Every proposal, score and verdict goes into a lab notebook. Every prose line is checked: a number not in the evidence is dropped.

The curve, from room data alone



The model reads the plant: menu labels into mechanisms

Given only the labels on the controller's menus (no values) and operator notes, the model wrote complete control laws. The fitter decides whether the data support them.

FROM THE PLANT	PROGRAM THE MODEL WROTE	ΔBIC VS BLIND BEST
"Curve value"	`base + slope * T_out`	+9.7
"Min limit"	`max(base + slope * T_out, mn)`	+14.1
"Autumn drying (a correction term)"	`base + slope * T_out + where(month() >= dry, boost, OFF)`	+16.3
"Outdoor temperature delay (a correction term)"	`base + slope * lag(T_out, delay)`	+11.0

Spot readings fix what room data cannot

MODE	ERROR VS OTHER PANEL READINGS	ROOM ERROR
HA-ONLY	12.33 °C	0.715 °C
ANCHORED k=1	7.09 °C	0.716 °C
ANCHORED k=2	6.03 °C	0.721 °C
ANCHORED k=3	2.19 °C	0.704 °C
ANCHORED k=4	2.23 °C	0.722 °C

Which floor does loop L1 heat? Nobody knew. L1 panel readings fit Upstairs floor heating best; margin 0.084 C of room error.

Blind evaluation against the configured curve

Read off the controller by hand, it entered the repository after the fits. Only evaluate.py reads it; the coding agent saw error numbers only.

Configured curves vs the panel's own readings	L1 0.46 °C, L2 0.49 °C - the file is consistent
HA-ONLY, identified curve vs configured L1	15.9 °C
ANCHORED k=3, identified curve vs configured L1	2.9 °C
Which floor is L1?	Spot-reading test: Upstairs floor heating (margin 0.084 °C). Evaluation by slope: Downstairs radiators (fireplace+tv) (margin 1.08 °C/°C). The two disagree: not settled.

AMD token factory: which model can do this job?

Every chat model on the endpoint, on Käyrä's own tasks, graded by code: programs through the validator, an operator note into a program, prose through the number guardrails.

MODEL	SCORE	VALID PROGRAMS	OPERATOR	GROUNDED PROSE	MEDIAN S	$ / CALL
DeepSeek-V4.1-Flash	0.96	88 %	100 %	100 %	13.0	0.00027
MiMo-V2.6-Flash	0.96	88 %	100 %	100 %	14.4	0.00042
DeepSeek-V4-Flash	0.75	25 %	100 %	100 %	8.5	0.00014
Gemma-4-31B-IT	0.74	71 %	50 %	100 %	17.1	0.00003
Qwen3.8-Flash-Next	0.67	0 %	100 %	100 %	70.5	0.00211
MiniCPM5-2B	0.46	38 %	0 %	100 %	47.0	0.00028

All runs used DeepSeek-V4.1-Flash (best at programs; on prose every model scored the same). Real-data run on AMD: ONLINE (AMD 13 calls, no fallback), total AMD cost $0.0119.

What worked, what did not

- Synthetic controllers in the real building model, three arms. Lowest held-out error: language model 5/6, grammar search 5/6, catalogue 4/6. Right mechanism by the checklist fixed before the runs: 1, 1, 1 of 6. Between two runs the model arm went from best in four cases to best in one: six cases is too few to claim more.
- Integrating controllers do not show in room data that sits near setpoint.
- The uncertainty band is too narrow. Room data alone get the slope, not the level.
- The model's prose: numbers are checked, reasoning is not. The computed tables are shown next to it.

AMD and what is next

- Language model: AMD token factory for every hypothesis, the site-context programs and the write-up. Model picked by the AMD benchmark above. Run: ONLINE (AMD 13 calls, no fallback).
- Fitter: PyTorch, 163,840 candidates in 31.86 s on a laptop CPU. AMD GPU: not measured, no estimate given.
- Next: OPC UA, Modbus and historian connectors, domain catalogues, the replacement controller, editing the setup in the panel.

github.com/apn201/kayra